

HPCプログラミングセミナー

GPUプログラミング入門（OpenACC編）

第1講: GPUアーキテクチャ基礎

高度情報科学技術研究機構

Research Organization for Information Science and Technology (RIST)

質問は随時受け付けます

E-mail (hpc-seminar@rist.or.jp) もご利用ください

この資料は、HPCプログラミングセミナー「アクセラレータ入門」として実施していたものと同じ内容です
OpenACCを用いたGPUプログラミングの基礎を解説するという資料の主旨を明確にするため、タイトルを変更しました

本資料の教育目的等での転載、複製及び再配布をご希望される場合は資料末尾の記述を参照ください

本講の目的

- GPUでアプリケーションを高速に実行するため、最小限把握しておくべきGPUアーキテクチャの基礎を理解する
 - ▶ 2講目（OpenACCでのGPU実装と最適化基礎）の理解に繋がる

自学自習のために

■ サンプルコードも用意（ご自由にお使いください）

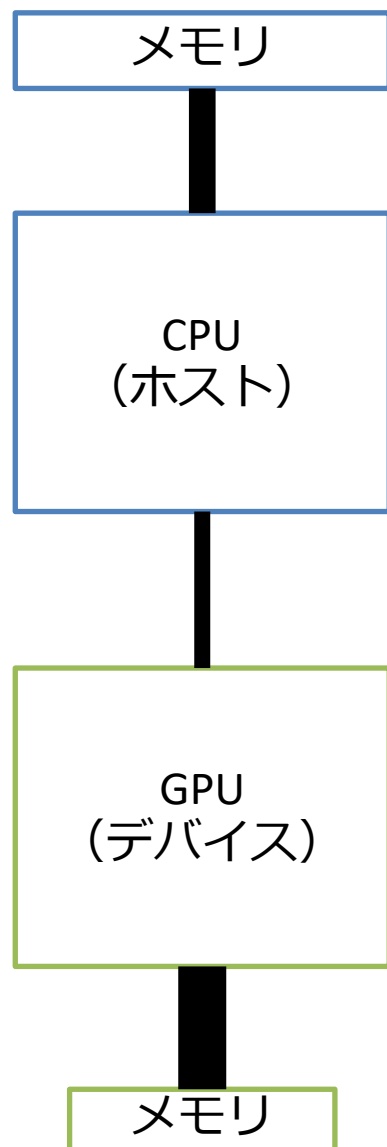
- ▶ OpenACCを中心に、OpenMP Target, Kokkosのサンプルを含む
- ▶ HPCIポータルサイトから入手可能
 - ▶ https://www.hpci-office.jp/events/seminars/seminar_texts

おことわり

- 本講はGPUアーキテクチャの基礎の内、代表的な物のみ取り上げています
 - ▶ 詳細は参考文献を参照ください
- 基礎ですが一部発展的な内容も載せています
 - ▶ (発展) と記しています (基礎学習では飛ばして構いません)
- 個別の事例については別途相談頂くか、HPCIヘルプデスクを通じてお問い合わせください
 - ▶ 現在のHPCIの利用の有無は関係ございませんのでお気軽にどうぞ

<http://www.hpci-office.jp/pages/helpdesk/>

GPUコンピューティングの概要



- 従来のCPU + アクセラレータ（演算加速器）としてのGPU
 - ▶ホスト（CPU）とデバイス（GPU）
 - ▶全ての処理はホスト側から指示
 - ▶GPUが得意な処理（多数データの演算など）を分離（オフロード）してGPUに担当させる
- 異なるハードウェアを組み合わせる故の特徴
 - ▶物理的に独立したメモリをそれぞれが持つ
 - ▶オフロードの際、ホストメモリ - デバイスメモリ間でのデータ転送が必須となる

処理のオフロードをCode上で記述する

OpenACCでの実装例

```
program main
  implicit none
  integer :: i
  integer, parameter :: n=64
  real :: a(n), b(n)
```

```
do i = 1, n
  a(i) = i
  b(i) = i*10
end do
```

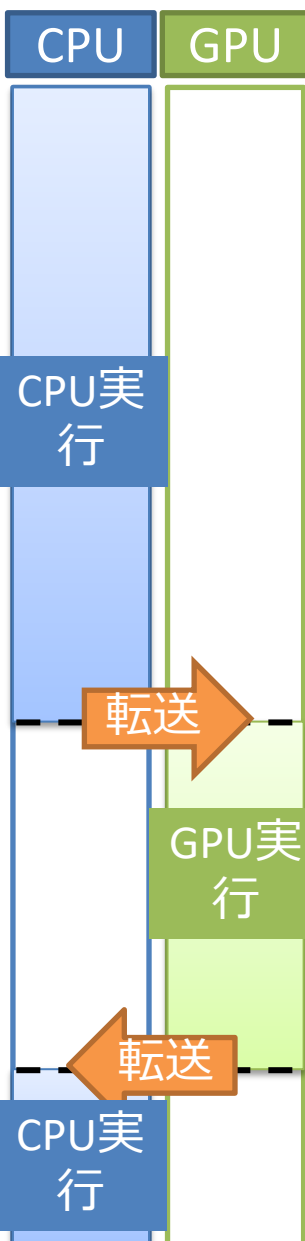
!\$acc kernels loop

```
do i = 1, n
  a(i) = a(i) + 2.0
  b(i) = b(i) * 2.0
end do
```

!\$acc end kernels

```
print *, a(:), b(:)
```

```
end program
```



■ OpenACCでGPU実装した例

- ▶ 指示行挿入のみで実装可能

■ ホストとデバイスでの処理を一つのコード内に記述可能

- ▶ 実行はホスト（CPU）から開始

- ▶ 処理は分離されてGPU向けの並列化が施される

- ▶ デバイスメモリでのデータの確保、ホストデバイス間でのデータ転送が行われる

CPU/GPUが並行に実行（コンピューティング）

CPUとGPU

GPUに合う処理をオフロードさせるため、
CPUとGPUのハードウェアの特徴を把握する必要がある

CPU

- クロック数の高い演算コアを少数もつ
 - ▶ 例: クロック数が2.4GHzのコアを96個
 - ▶ 1命令で複数データを同時に処理可能（例：Intel AVX）
- 構造は複雑化
 - ▶ アウトオブオーダー等、無駄な待ち時間を省く機能を備える
 - ▶ シリアル処理を高速処理可能
 - ▶ 結果として演算器の割合少なめ
 - ▶ チップあたりの演算性能はGPUより低い
- 並列化が難しい処理 or 低い並列性の処理が得意

GPU

- クロック数の低い演算コアを多数もつ
 - ▶ 例: クロック数が1.98GHzのコアを16896個
 - ▶ 特定の演算（行列演算など）専用演算器も搭載
- 構造は単純
 - ▶ アウトオブオーダー等の演算を効率化する補助機能少ない
 - ▶ シリアル処理は不得意
 - ▶ 演算器に多くのリソースを割いている
 - ▶ チップあたりでの高い演算性能を実現
- 多数のコアを活かした、高い並列性の処理が得意

CPUとGPUのハードウェアとしての比較

東京大学Wisteria-A (Aquarius)に搭載されているCPU・GPUのハードウェア仕様

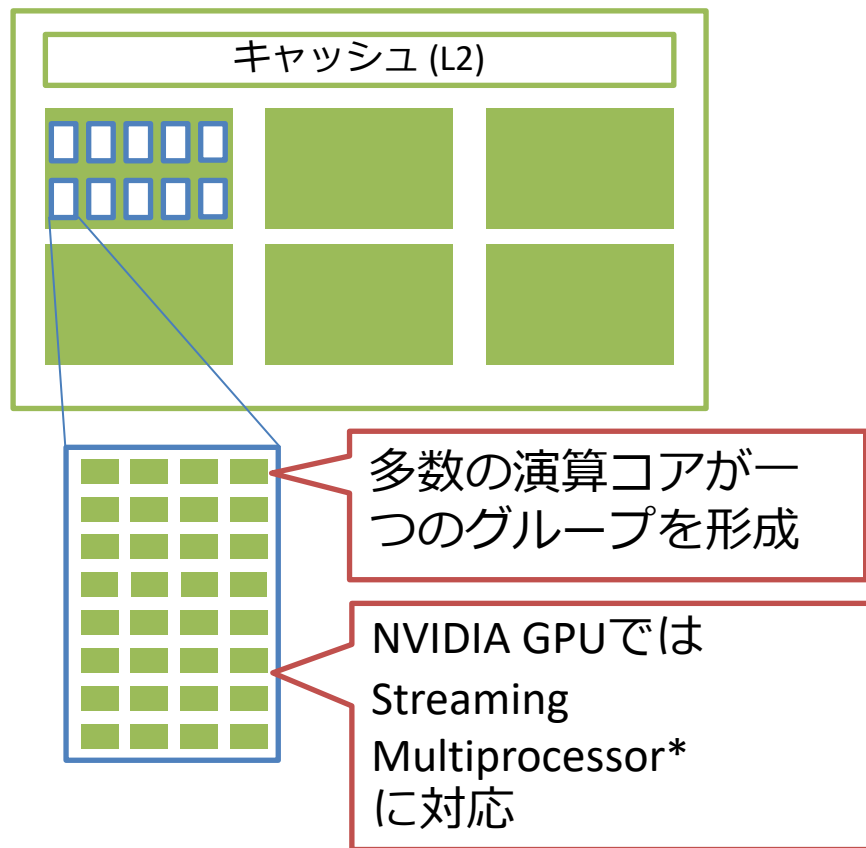
	Intel Xeon 8360Y	NVIDIA Tesla A100
FP64演算コア数	36	3456
FP32演算コア数		6912
理論演算性能 (FP32) [TFLOPS]	2.8	19.5
理論演算性能 (FP64) [TFLOPS]		9.7
メモリバンド幅[GB/s]	205	1555

<https://www.cc.u-tokyo.ac.jp/supercomputer/wisteria/system.php>

<https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>

GPUの構造（もう少し整理）

GPUのハード構造の模式図



* [備考] OpenCLやHIPでは? → 補足:
GPUのterminologyを参照

- 多数のコアで並列計算を行う
 - 逐次処理では大半の演算コアが無駄
 - 独立な処理を重ねて待ち時間の隠蔽を狙う
 - 十分な並列性を持った並列処理が必要
- コアはグループ分けされた階層構造をとる
 - 階層ごとに特徴あり（キャッシュの扱いやスレッド間同期処理など）
 - 各階層への処理の割り当て方で性能が変化
 - OpenACCでは?
 - 基礎: コンパイラに割り当てを任せる
 - 発展: ユーザが操作して最適化する

GPUでの並列計算の基本

- 処理の最小単位: スレッド
- 多数のコアを使い切るためデータ並列が必須
 - ▶ 並列処理の典型例
 - ▶ 1データを1スレッドに割り当てる
 - ▶ 1スレッドを1コアに割り当てる
 - ▶ スレッドを並列に処理させる
 - ▶ スレッド数 >> コア数で実行
 - ▶ 「処理が終了したらスレッドをコアから外す→新しいスレッドをコアに割り当てる」を繰り返す
 - ▶ コアを定常的にアクティブにする
- 複数データに同じ命令を同時に発行
 - ▶ NVIDIA GPUの場合: 連続する32スレッド(warp)が1つのまとまり→実質的な処理の最小単位

```
for(int i = 0; i<N; i++)  
{  
    A[i] = B[i] + C[i];  
}
```

コアとスレッドとデータへの割り当て例

コア0 / スレッド0	:	A[0]	=	B[0]	+	C[0]
コア1 / スレッド1	:	A[1]	=	B[1]	+	C[1]
コア2 / スレッド2	:	A[2]	=	B[2]	+	C[2]
.....						
.....						
コア29 / スレッド29	:	A[29]	=	B[29]	+	C[29]
コア30 / スレッド30	:	A[30]	=	B[30]	+	C[30]
コア31 / スレッド31	:	A[31]	=	B[31]	+	C[31]

GPUの並列計算にて最も重要なこと

- GPUが持つ高い性能を引き出すには「スレッド数（データ数）>>コア数」が必要
 - ▶ 例: NVIDIA A100の6912個に対して最低数万スレッド
- 演算リソースの利用効率を上げることが必要
 - ▶ 多数の演算コアを遊ばせない
 - ▶ メモリアクセスなどの演算に向けた待ち時間を無駄にしない
 - ▶ 例: 演算を行っているスレッドの裏で他のスレッドがメモリアクセスなどを行う
 - ▶ 演算が終わったらすぐに他のスレッドをコアに割り当てることが可能になる
- アプリケーション側の検討事項: 以下2点に配慮する
 - ▶ 入力値のサイズ設定
 - ▶ 高並列計算が可能なアルゴリズムの採用

CPUよりも要求
要件高い

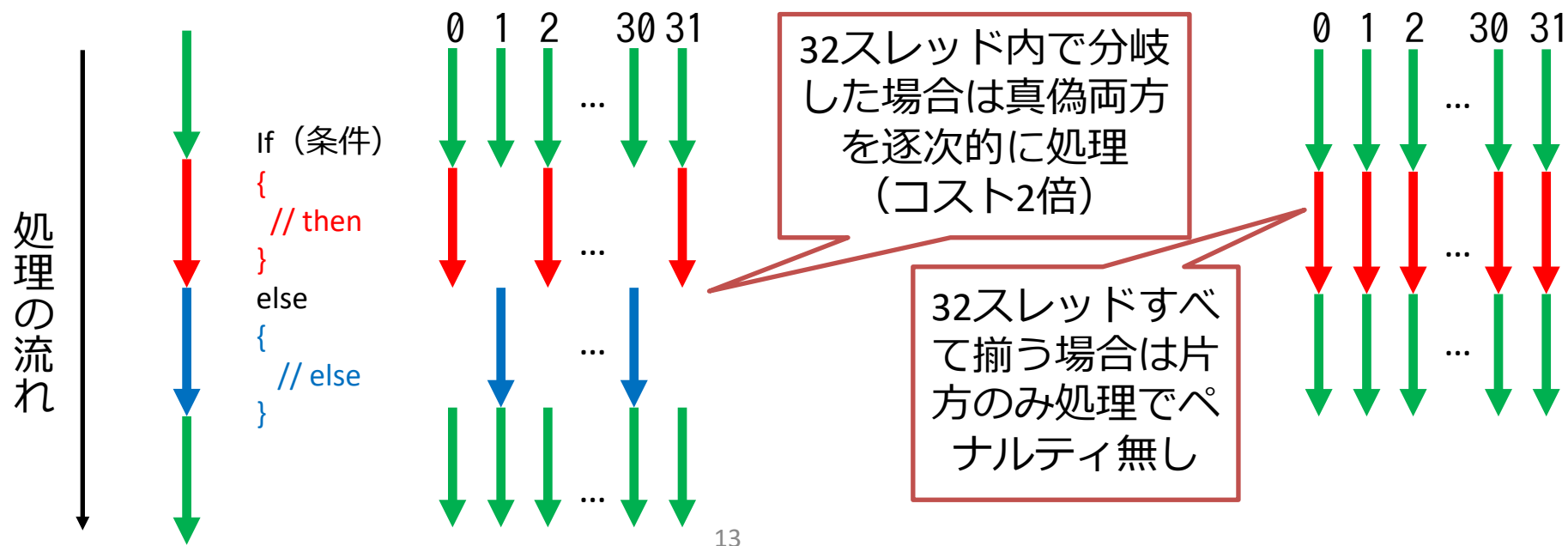
GPU利用時にその他考慮すること

1. 処理の分岐
2. メモリアクセス
3. GPUへのオフロードに伴う転送コスト
4. スレッドの階層（発展）

処理の分岐（IF文など）は注意！

■ 連続した32スレッドに同じ命令を発行する

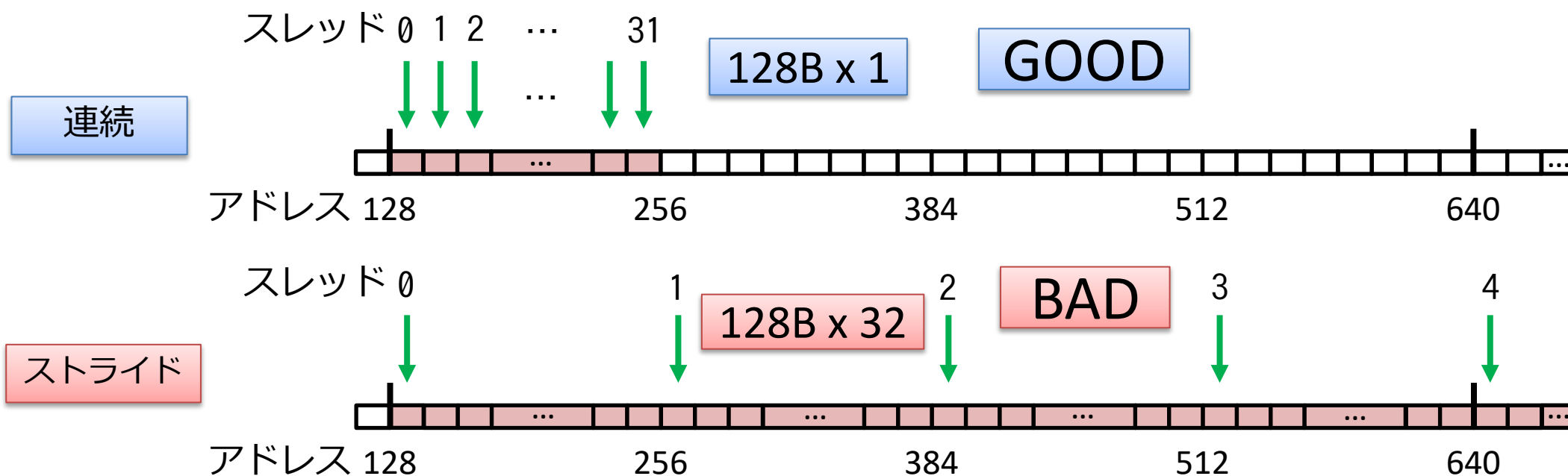
- ▶ 分岐の先ごとに命令を発行する可能性
 - ▶ 例: 32スレッドのうち, 16スレッドが真, 16スレッドが偽
- ▶ アルゴリズムとして, なるべく避けるべき
 - ▶ 不可避の場合: 分岐する処理量を減らす



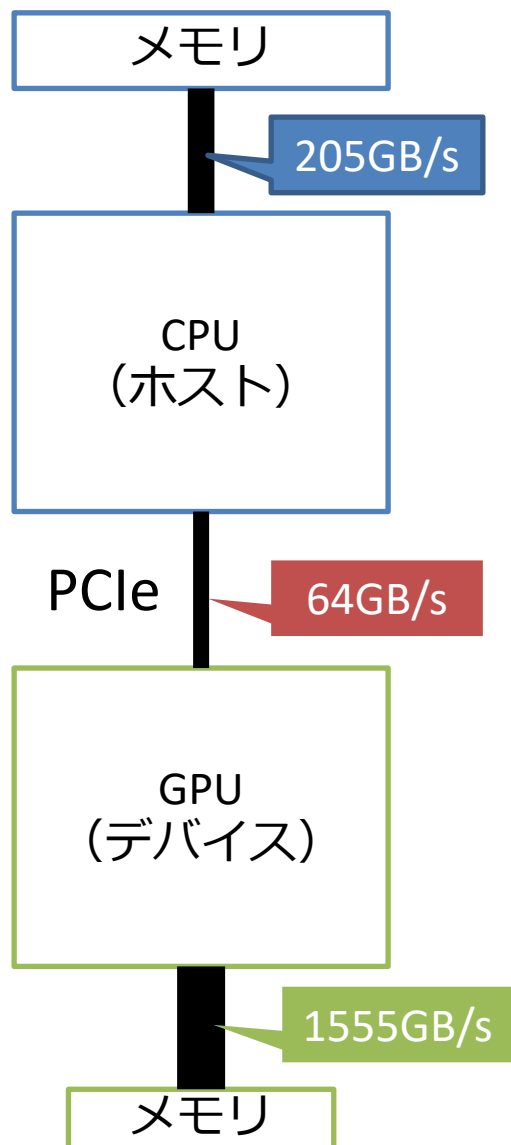
GPUのメモリアクセス

■ メモリアクセスは連続した領域に対して行うこと

- ▶ GPUのメモリでは128 B(Bytes)単位によるメモリアクセスの管理が基本
- ▶ ストライドアクセスの場合：128 B単位を複数読み込んで効率悪い
- ▶ CPUでも起こる問題だが、GPUではより注意すること



GPUのメモリ オフロード故の問題点



- PCI-Express (PCIe) の転送幅はメモリバンド幅よりも一桁小さい
 - ▶ 不要な転送は性能のボトルネックになる
 - ▶ 実装方法の見直し、アルゴリズム改善で削減
 - ▶ 他の処理と非同期に実行して隠蔽する（発展）
- ▶ 各々が個別のメモリ空間を持つ
 - ▶ ホストとデバイスで物理的に別のメモリを持つ
 - ▶ ユーザが意識して管理する必要あり
 - ▶ 同一に扱う技術もある（NVIDIA GPUでのユニファイドメモリ）が、適用範囲に制限あり
- ▶ これらの問題へのOpenACCでの対応のやり方は第2講で紹介

多数スレッドを階層に配置

- GPUのハードウェアは階層的な構造で、コアがグループ分けされている
- スレッドもグループ分けされた上で処理される
- OpenMP : 「コア数 = スレッド数」で (最外) ループ分割が一般的
 - ▶ SIMD処理と合わせると2階層
- OpenACC : 3階層 (gang, worker, vector)
 - ▶ gang=16, worker=4, vector=128などと設定
 - ▶ 総スレッド数 = gang × worker × vector
 - ▶ NVIDIAのGPUでworkerの階層を気にする必要は少ない
 - ▶ 例 : 多重ループで最外ループはgang, 2番目はworker, 3番目はvector分割
 - ▶ この3階層をハードの階層構造に当てはめて処理
 - ▶ ハードウェアのリソース(キャッシュ等)の共有範囲、同一命令発行単位の幅などに影響
- どの階層にどれだけ割り当てるか？
 - ▶ CUDA : ユーザが設定 (grid, block) ※設定の基本的な方針は存在する
 - ▶ OpenACC : コンパイラが設定 (基本それで十分)、ユーザが設定 (最適化) 可能だが、CUDAの活用と同レベルの理解が必要

GPUに向いているアプリケーション

- メモリアクセスに比べて演算の割合が大きいもの
 - ▶ GPUの高い演算性能を活用できる
 - ▶ (発展) 演算強度、ループラインモデルなどを参考に性能見積もり
- データ並列性が高いもの
 - ▶ 多数ある演算コアを遊ばせない
 - ▶ 「演算」「メモリアクセスなどの演算に向けた処理」を並行させる
- 同一の命令を連続したデータに発行するもの
 - ▶ NVIDIA GPUは32スレッドが実質的な処理の最小単位 (Warp)
- 規則的なメモリアクセスを行うもの
- データの局所性 (ホスト-デバイス間) が高いもの
 - ▶ 不要なデータ転送は性能のボトルネックになる

Cf. AMD GPUでは64スレッドが実質的な処理の最小単位 (Wavefront)

以上を満たす処理が多く含まれるなら、GPUの利用をおすすめします
参考：OpenACC Success Stories, <https://www.openacc.org/success-stories>

GPUアーキテクチャ基礎のまとめ (1/2)

- ホスト（CPU）からデバイス（GPU）に処理をオフロード
 - ▶ GPUに適した処理を優先してオフロードする必要
 - ▶ CPUとGPUの処理を並行させて効率化（第3講で説明）
- GPUは階層化した多数の演算コアを有する
 - ▶ 並列計算は必須、シリアル処理は避ける
- GPU・CPUで別々のメモリを持つ
 - ▶ 別々にメモリを確保し、両者間で転送する
 - ▶ PCIeはメモリバンド幅より一桁小さい
 - ▶ 実装方法の見直し、アルゴリズム改善で無駄な転送を削減
 - ▶ 他の処理と非同期に実行して隠蔽する（発展）

GPUアーキテクチャ基礎のまとめ（2/2）

■十分な大きさのデータ並列性が必要

- ▶ 多数(>1000)の演算コアの活用とメモリアクセスなどの待ち時間を効率的に隠蔽するため

■その他

- ▶ 同一命令が発行されるスレッド群での分岐の扱い
- ▶ メモリへの連続したアクセスの必要性

■GPUのハードウェアとスレッドの階層構造の対応（発展）

- ▶ スレッドは3階層（gang, worker, vector）でグループ分け
- ▶ OpenACCではコンパイラ任せで十分（基礎の範囲内では）

第1講のまとめ

■ GPUアーキテクチャの基礎を紹介しました

- ▶ 処理のオフロード
- ▶ CPU・GPU間でのデータ転送
- ▶ GPUでの並列計算

■ 第2講ではこれらの特徴を踏まえた上で、OpenACCを用いてアプリケーションをGPU実装する方法と最適化の基礎を紹介します

実験: NVIDIA GPUの仕様を確認する (1/2)

■ おすすめのコマンド (高優先度)

- ▶ `nvaccelinfo` (NVIDIA HPC SDKの利用が前提)
- ▶ `nvidia-smi -L` → 利用可能なGPU数の確認に便利
- ▶ `nvidia-smi topo -m` → CPU-GPU, GPU-GPUの接続情報の確認に便利

■ 自分で情報取得用のコードを書く (低優先度)

- ▶ OpenACC/OpenMPのruntime library APIを使用する。
 - ▶ 詳細なハードウェア情報の取得は難しい
 - ▶ CUDAで書くほうがよい (参考文献[5]を参照)

実験: NVIDIA GPUの仕様を確認する (2/2)

```
CUDA Driver Version:      12020
NVRM version:             NVIDIA UNIX x86_64 Kernel Module  535.230.02  Fri Dec 20 21:42:05 UTC 2024

Device Number:            0
Device Name:              NVIDIA A100 80GB PCIe
Device Revision Number:   8.0
Global Memory Size:       84987740160
Number of Multiprocessors: 108
Concurrent Copy and Execution: Yes
Total Constant Memory:    65536
Total Shared Memory per Block: 49152
Registers per Block:      65536
Warp Size:                32
Maximum Threads per Block: 1024
Maximum Block Dimensions: 1024, 1024, 64
Maximum Grid Dimensions:  2147483647 x 65535 x 65535
Maximum Memory Pitch:     2147483647B
Texture Alignment:        512B
Clock Rate:               1410 MHz
Execution Timeout:        Yes
Integrated Device:        No
Can Map Host Memory:      Yes
Compute Mode:             default
Concurrent Kernels:       Yes
ECC Enabled:              Yes
Memory Clock Rate:        1512 MHz
Memory Bus Width:         5120 bits
L2 Cache Size:            41943040 bytes
Max Threads Per SMP:      2048
Async Engines:            3
Unified Addressing:       Yes
Managed Memory:          Yes
Concurrent Managed Memory: Yes
Preemption Supported:     Yes
Cooperative Launch:      Yes
Default Target:           cc80
```

nvaccelinfoの出力例
(第3講でも触れる)

GPUのメインメモリ (グローバルメモリ)の容量 (bytes)

Streaming multiprocessor (SM)の数
→ SMの中に多数の演算コアがある

連続した1つのまとまりとして発行されるスレッド数

L2キャッシュのサイズ (コアで共有)

CUDA Driver VersionとDefault TargetはOpenACCを使う場合でも大事な情報(コンパイル時に把握しておくことを推奨)

補足: GPUのterminology

■ CUDA (NVIDIA) , OpenCL, HIP (ROCm)の間で用語を比較

CUDA (NVIDIA)	OpenCL	HIP (ROCm)	OpenACC	OpenMP Offload
Streaming multiprocessor	Compute Unit	Compute Unit		
CUDA Core	Processing Elements	Processing Elements		
Global memory	Global memory	Global memory		
Shared memory	Local memory	Shared memory		
Grid	NDRange	Grid	gang	league
Thread block	Work-group	Thread block	worker, vector	thread team
CUDA thread	Work-item	Threads		
Warp	Sub-group	Wavefront or Warp		

T. Deakin and T. G. Mattson, “Programming your GPU with OpenMP: Performance Portability for GPUs” (2023, MIT) Chap.1.
G. M. Thomas, “Advanced GPU Assembly Programming: A Technical Reference for NVIDIA and AMD Architectures” (2024).
<https://docs.nvidia.com/cuda/index.html>
<https://registry.khronos.org/OpenCL/>
https://rocm.docs.amd.com/projects/HIP/en/latest/understand/programming_model.html

参考文献

1. HPC WORLD, 「OpenACCによるプログラミング」, <https://hpcworld.jp/archive/SPG/Pgi/OpenACC/>
2. Rob Farber, “Parallel Programming with OpenACC” (MK, 2016).
3. John Cheng, Max Grossman, Ty McKercher (訳: 株式会社クイープ, 監訳: 森野慎也, 成瀬彰), 「CUDA C プロフェッショナルプログラミング」 (インプレス, 2015)
4. Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj, “Programming Massively Parallel Processors: A Hands-on Approach” 4th ed (MK, 2022)
5. OpenACC Getting Started Guide, <https://docs.nvidia.com/hpc-sdk/compiler/openacc-gs/index.html>
6. CUDA C++ Programming Guide; <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

謝辞

- 本資料（サンプルコード、計測）の一部において、東京科学大学のスーパーコンピュータTSUBAME4.0を利用しました

本資料は、構成・文章・画像などの全てにおいて著作権法上の保護を受けています。以下の条件で本資料の利用を許可します。

- 本資料の利用に際し、著作者への連絡は不要ですが、著作者のクレジット（当機構の名称）は明示して下さい。
- 本資料の一部あるいは全部について、転載、複写及び再配布を許可します。ただし、本資料を改変した場合は除きます。
- 本資料を販売するなどの営利目的での利用は禁じます。教育目的等で利用することは産・学・官を問わず許可します。

※上記はクリエイティブ・コモンズ「表示-非営利-改変禁止 4.0 国際（CC BY-NC-ND 4.0）」相当です。

- 本資料に記載された内容などは、予告なく変更される場合があります。
- 本資料に起因して使用者に直接または間接的損害が生じても、著作者はいかなる責任も負わないものとします。

なお、本資料の旧版についても上記と同様の条件で利用することを許可します。

ご不明の点があれば下記のヘルプデスクにお問い合わせ下さい。

ヘルプデスク： helpdesk[-at-]hpci-office.jp（[-at-]を@にしてください）

